МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

Федеральное государственное автономное образовательное учреждение высшего образования
«Московский Авиационный Институт»
(Национальный Исследовательский Университет)

Институт: №8 «Компьютерные науки и прикладная математика»

Лабораторная работа № 1 по курсу «Численные методы»

Группа: М8О-303Б-23

Студент(ка): Жуховицкий А. Д.

Преподаватель: Ревизников Д.Л.

Оценка:

Москва, 2026

ОГЛАВЛЕНИЕ

1 Тема.....3
2 Задание 3
3 Ход лабораторной работы.....4

1 Тема

Численные методы линейной алгебры

2 Задание

1.1. Реализовать алгоритм LU - разложения матриц (с выбором главного элемента) в виде программы. Используя разработанное программное обеспечение, решить систему линейных алгебраических уравнений (СЛАУ). Для матрицы СЛАУ вычислить определитель и обратную матрицу.

$$7.\ \begin{cases} x_1 - 5 \cdot x_2 - 7 \cdot x_3 + x_4 = -75 \\ x_1 - 3 \cdot x_2 - 9 \cdot x_3 - 4 \cdot x_4 = -41 \\ -2 \cdot x_1 + 4 \cdot x_2 + 2 \cdot x_3 + x_4 = 18 \\ -9 \cdot x_1 + 9 \cdot x_2 + 5 \cdot x_3 + 3 \cdot x_4 = 29 \end{cases}$$

1.2. Реализовать метод прогонки в виде программы, задавая в качестве входных данных ненулевые элементы матрицы системы и вектор правых частей. Используя разработанное программное обеспечение, решить СЛАУ с трехдиагональной матрицей.

$$7.\ \begin{cases} 15 \cdot x_1 + 8 \cdot x_2 = 92 \\ 2 \cdot x_1 - 15 \cdot x_2 + 4 \cdot x_3 = -84 \\ 4 \cdot x_2 + 11 \cdot x_3 + 5 \cdot x_4 = -77 \\ -3 \cdot x_3 + 16 \cdot x_4 + -7 \cdot x_5 = 15 \\ 3 \cdot x_4 + 8 \cdot x_5 = -11 \end{cases}$$

1.3. Реализовать метод простых итераций и метод Зейделя в виде программ, задавая в качестве входных данных матрицу системы, вектор правых частей и точность вычислений. Используя разработанное программное обеспечение, решить СЛАУ. Проанализировать количество итераций, необходимое для достижения заданной точности.

7. $$\begin{cases} 29 \cdot x_1 + 8 \cdot x_2 + 9 \cdot x_3 - 9 \cdot x_4 = 197 \\ -7 \cdot x_1 - 25 \cdot x_2 + 9 \cdot x_4 = -226 \\ x_1 + 6 \cdot x_2 + 16 \cdot x_3 - 2 \cdot x_4 = -95 \\ -7 \cdot x_1 + 4 \cdot x_2 - 2 \cdot x_3 + 17 \cdot x_4 = -58 \end{cases}$$

1.4. Реализовать метод вращений в виде программы, задавая в качестве входных данных матрицу и точность вычислений. Используя разработанное программное обеспечение, найти собственные значения и собственные векторы симметрических матриц. Проанализировать зависимость погрешности вычислений от числа итераций.

7. $$\begin{pmatrix} -6 & 6 & -8 \\ 6 & -4 & 9 \\ -8 & 9 & -2 \end{pmatrix}$$

1.5. Реализовать алгоритм QR – разложения матриц в виде программы. На его основе разработать программу, реализующую QR – алгоритм решения полной проблемы собственных значений произвольных матриц, задавая в качестве входных данных матрицу и точность вычислений. С использованием разработанного программного обеспечения найти собственные значения матрицы.

7. $$\begin{pmatrix} 9 & 0 & 2 \\ -6 & 4 & 4 \\ -2 & -7 & 5 \end{pmatrix}$$

3 Ход лабораторной работы

LUP-разложение матрицы A с перестановкой строк, решение системы $Ax = b$, вычисление определителя и обратной матрицы.

```
import numpy as np
import pandas as pd
from IPython.display import display

from mtx_utils import load_mtx

A, b = load_mtx("tasks/1_1.txt")
n = len(b)

A, b
```

```
(array([[ 1., -5., -7.,  1.],
        [ 1., -3., -9., -4.],
        [-2.,  4.,  2.,  1.],
        [-9.,  9.,  5.,  3.]]),
 array([-75., -41.,  18.,  29.]))
```

$$Ax = b,$$
$$P^{-1}LU = A.$$

```
# Permutations для строк в матрице
P = np.eye(n, dtype=float)

L = np.eye(n, dtype=float)
U = A.copy()
swap_count = 0

for col in range(n):
    pivot_row = col + np.argmax(np.abs(U[col:, col]))

    if np.isclose(U[pivot_row, col], 0.0):
        raise ValueError("Матрица вырожденная: LU-разложение невозможно")

    if pivot_row != col:
        swap_count += 1
        U[[col, pivot_row], :] = U[[pivot_row, col], :]
        P[[col, pivot_row], :] = P[[pivot_row, col], :]

        L[[col, pivot_row], :col] = L[[pivot_row, col], :col]

    for row in range(col + 1, n):
        factor = U[row, col] / U[col, col]
        L[row, col] = factor
        U[row, col:] -= factor * U[col, col:]
```

```
display(pd.DataFrame(U), pd.DataFrame(L))
```

	0	1	2	3
0	-9.0	9.0	5.000000	3.000000
1	0.0	-4.0	-6.444444	1.333333
2	0.0	0.0	-5.222222	-4.333333
3	0.0	0.0	0.000000	2.936170

	0	1	2	3
0	1.000000	0.0	0.000000	0.0
1	-0.111111	1.0	0.000000	0.0
2	-0.111111	0.5	1.000000	0.0
3	0.222222	-0.5	0.446809	1.0

$$Ux = z,$$
$$Lz = Pb.$$

```
from IPython.display import Math

pb = P @ b

z = np.zeros(n, dtype=float)
for row in range(n):
    z[row] = pb[row] - np.dot(L[row, :row], z[:row])

x = np.zeros(n, dtype=float)
for row in range(n - 1, -1, -1):
    s = np.dot(U[row, row + 1:], x[row + 1:])
    x[row] = (z[row] - s) / U[row, row]

for i in range(n):
    display(Math(fr"x_{i+1} = {x[i]:.3f}"))
```

$x_1 = 2.000$

$x_2 = 4.000$

$x_3 = 7.000$

$x_4 = -8.000$

```
detA = (-1) ** swap_count * np.prod(np.diag(U))
display(Math(fr"det A = {detA:.3f}"))
```

$detA = 552.000$

$$
\begin{aligned}
PA &= LU, \\
PAA^{-1} &= LUA^{-1}, \\
P &= LUA^{-1}.
\end{aligned}
$$

Обозначим

$$
\begin{aligned}
C &= UA^{-1}, \\
LC &= P.
\end{aligned}
$$

```
A_inv = np.zeros((n, n), dtype=np.float64)

C = np.zeros((n, n), dtype=np.float64)
for row in range(n):
    C[row, :] = P[row, :] - L[row, :row] @ C[:row, :]

for row in range(n - 1, -1, -1):
    A_inv[row, :] = (C[row, :] - U[row, row + 1:] @ A_inv[row + 1:, :]) / U[row, row]

pd.DataFrame(A_inv)
```

	0	1	2	3
0	0.028986	-0.032609	0.525362	-0.228261
1	0.007246	0.054348	0.568841	-0.119565
2	-0.108696	-0.065217	-0.282609	0.043478
3	0.246377	-0.152174	0.340580	-0.065217

Проверка решения

```
np.allclose(x, np.linalg.solve(A, b))
```

```
True
```

Проверка корректности матриц L U и перестановки строк

```
np.allclose(A, P.T @ L @ U)
```

```
True
```

Проверка корректности вычисленного определителя

```
np.isclose(np.linalg.det(A), detA)
```

```
np.True_
```

Проверка корректности обратной матрицы

```
np.allclose(np.linalg.inv(A), A_inv)
```

```
True
```

Метод прогонки

```
import numpy as np

slau = np.matrix(
    [
        [0, 15, 8],
        [2, -15, 4],
        [4, 11, 5],
        [-3, 16, -7],
        [3, 8, 0]
    ]
)

d = [92, -84, -77, 15, -11]

n = 5
P, Q = np.zeros(n + 1, dtype=float), np.zeros(n + 1, dtype=float)

P[0], Q[0] = 0, 0

for i in range(0, n):
    P[i + 1] = -slau[i, 2] / (slau[i, 1] + slau[i, 0] * P[i])
    Q[i + 1] = (d[i] - slau[i, 0] * Q[i]) / (slau[i, 1] + slau[i, 0] * P[i])
```

```
from IPython.display import display, Math

x = np.zeros(n, dtype=float)
for i in range(n, 0, -1):
    Pi = f"{P[i]:.3f}"
    Qi = f"{Q[i]:.3f}"
    if P[i] == 0:
        x[i - 1] = Q[i]
        display(Math(fr"x_{i} = {Qi}"))
        continue
    x[i - 1] = P[i] * x[i] + Q[i]
    xi = f"{x[i - 1]:.3f}"
    display(Math(fr"x_{i} = {xi}"))
```

$x_5 = -1.000$

$x_4 = -1.000$

$x_3 = -8.000$

$x_2 = 4.000$

$x_1 = 4.000$

```
check = np.matrix(
    [
        [15, 8, 0, 0, 0],
```

```
        [2, -15, 4, 0, 0],
        [0, 4, 11, 5, 0],
        [0, 0, -3, 16, -7],
        [0, 0, 0, 3, 8]
    ]
)

print(np.linalg.solve(check, d)[::-1])
```

```
[-1. -1. -8.  4.  4.]
```

```
import numpy as np

from mtx_utils import load_mtx

A, b = load_mtx("tasks/1_3.txt")
n = len(b)

A, b
```

```
(array([[ 29.,   8.,   9.,  -9.],
        [ -7., -25.,   0.,   9.],
        [  1.,   6.,  16.,  -2.],
        [ -7.,   4.,  -2.,  17.]]),
 array([ 197., -226.,  -95.,  -58.]))
```

Введём функции нормы

```
def matrix_norm(matrix):
    matrix = np.asarray(matrix)
    return np.abs(matrix).sum(axis=1).max()

def vector_norm(vector):
    vector = np.asarray(vector)
    return np.abs(vector).max()
```

Точное решение СЛАУ:

```
from IPython.display import display, Math

actual = np.linalg.solve(A, b)
for i in range(n):
    display(Math(fr"x_{i + 1} = {actual[i]:.3f}"))
```

$x_1 = 7.000$

$x_2 = 6.000$

$x_3 = -9.000$

$x_4 = -3.000$

Метод простых итераций

Найдём α и β

```
import pandas as pd

alpha = A.copy()
beta = b.copy()
```

```
for i in range(n):
    beta[i] /= alpha[i, i]
    alpha[i] /= -alpha[i, i]
    alpha[i, i] = 0

display(pd.DataFrame(alpha), pd.DataFrame(beta))
```

	0	1	2	3
0	0.000000	-0.275862	-0.310345	0.310345
1	-0.280000	0.000000	0.000000	0.360000
2	-0.062500	-0.375000	0.000000	0.125000
3	0.411765	-0.235294	0.117647	0.000000

	0
0	6.793103
1	9.040000
2	-5.937500
3	-3.411765

Зададим точность $\varepsilon = 0.001$

```
eps = 1e-3
```

```
actual = np.linalg.solve(A, b)
for i in range(n):
    display(Math(fr"x_{i+1} = {actual[i]:.3f}"))
```

$x_1 = 7.000$

$x_2 = 6.000$

$x_3 = -9.000$

$x_4 = -3.000$

```
norm_alpha = matrix_norm(A)
norm_beta = vector_norm(b)

iteration = 1
prev_x = beta.copy()
```

```
while True:
    x_iteration = beta + alpha @ prev_x
    eps_i = vector_norm(x_iteration - prev_x)
    if norm_alpha < 1:
        eps_i *= norm_alpha / (1 - norm_alpha)
```

```
    if eps_i <= eps:
        break

    prev_x = x_iteration
    iteration += 1
```

```
for i in range(n):
    display(Math(fr"x_{i+1} = {x_iteration[i]:.3f}"))

print(f"{iteration} итераций")
```

$x_1 = 7.000$

$x_2 = 6.000$

$x_3 = -9.000$

$x_4 = -3.000$

17 итераций

Метод Зейделя

```
C, B = np.triu(alpha), np.tril(alpha, k=-1)
# верхняя, с главной диагональю и нижняя, без главной диагонали
norm_c = matrix_norm(C)

iteration = 1
prev_x = beta.copy()
```

```
while True:
    x_seidel = prev_x.copy()

    for i in range(n):
        x_seidel[i] = beta[i] + alpha[i] @ x_seidel

    eps_i = vector_norm(x_seidel - prev_x)
    if norm_c < 1:
        eps_i *= norm_c / (1 - norm_c)

    if eps_i <= eps:
        break

    prev_x = x_seidel
    iteration += 1
```

```
for i in range(n):
    display(Math(fr"x_{i+1} = {x_seidel[i]:.3f}"))

print(f"{iteration} итераций")
```

$x_1 = 7.000$

$x_2 = 6.000$

$x_3 = -9.000$

$x_4 = -3.000$

```
9 итераций
```

Проверка решения

```
np.allclose(x_seidel, x_iteration, eps)
```

```
True
```

```
np.allclose(x_seidel, actual, eps)
```

```
True
```

```
import numpy as np
import pandas as pd
from IPython.display import display, Math

from mtx_utils import load_mtx

eps = 1e-3
A, b = load_mtx("tasks/1_4.txt")
n = len(b)

A, b
```

```
(array([[-6.,  6., -8.],
        [ 6., -4.,  9.],
        [-8.,  9., -2.]]),
 array([0., 0., 0.]))
```

Метод вращения

```
def get_t(a_k_lower: np.ndarray):
    return np.sqrt(np.sum(a_k_lower ** 2))
```

```
Ak = A.copy()
a_k_lower = np.tril(Ak, k=-1)
iterations = 0
fi_k = 0

V = np.eye(n)

while get_t(a_k_lower) > eps:
    iterations += 1

    i, j = np.unravel_index(np.argmax(np.abs(a_k_lower)), a_k_lower.shape)
    fi_k = 0.5 * np.arctan2(2 * Ak[i, j], Ak[i, i] - Ak[j, j])

    Uk = np.eye(n)
    Uk[i, i] = Uk[j, j] = np.cos(fi_k)
    Uk[i, j] = -np.sin(fi_k)
    Uk[j, i] = np.sin(fi_k)

    Ak = Uk.T @ Ak @ Uk
    V = V @ Uk

    a_k_lower = np.tril(Ak, k=-1)
```

```
lambdas = np.diag(Ak)

for i, lam in enumerate(lambdas, start=1):
    display(Math(fr"\lambda_{{{i}}} = {lam:.3f}"))

print(f"Итераций: {iterations}")
```

$\lambda_1 = -19.344$

$\lambda_2 = 0.771$

$\lambda_3 = 6.573$

```
Итераций: 6
```

```
for i in range(n):
    vec = V[:, i]
    vec_str = r" \\ ".join(f"{x:.3f}" for x in vec)
    display(Math(fr"x^{{({i+1})}} = \begin{{pmatrix}} {vec_str} \end{{pmatrix}}"))
    print()
```

$$x^{(1)} = \begin{pmatrix} 0.596 \\ -0.567 \\ 0.569 \end{pmatrix}$$

$$x^{(2)} = \begin{pmatrix} 0.762 \\ 0.623 \\ -0.178 \end{pmatrix}$$

$$x^{(3)} = \begin{pmatrix} -0.253 \\ 0.540 \\ 0.803 \end{pmatrix}$$

Проверка

```
eigenvalues, eigenvectors = np.linalg.eig(A)

idx = np.argsort(eigenvalues)
eigenvalues = eigenvalues[idx]
eigenvectors = eigenvectors[:, idx]
```

```
idx = np.argsort(lambdas)
```

```
display(pd.DataFrame(lambdas).T)
```

	0	1	2
0	-19.344142	0.770697	6.573446

```
lambdas = lambdas[idx]
np.allclose(lambdas, eigenvalues, eps)
```

```
True
```

```
V[:, idx]
```

```
array([[ 0.59591222,  0.76204662, -0.25332505],
       [-0.56672685,  0.62257015,  0.5396546 ],
       [ 0.56895458, -0.17802067,  0.80286944]])
```

```
V = V[:, idx]
np.allclose(A @ V, lambdas * V, eps)
```

```
True
```

```
import numpy as np
import pandas as pd
from IPython.display import display, Math

from mtx_utils import load_mtx

eps = 1e-3
A, b = load_mtx("tasks/1_5.txt")
n = len(b)

A, b
```

```
(array([[ 9.,  0.,  2.],
        [-6.,  4.,  4.],
        [-2., -7.,  5.]]),
 array([0., 0., 0.]))
```

QR разложение

```
def get_householder(v: np.ndarray):
    n = v.shape[0]
    denom = v @ v
    if np.isclose(denom, 0):
        return np.eye(n)
    return np.eye(n) - 2 * np.outer(v, v) / denom
```

$$QR = A.$$

```
def get_qr(A: np.ndarray):
    Ai = A.copy()
    Q = np.eye(n)

    for i in range(n - 1):
        b = Ai[:, i]
        v = np.zeros(n)
        for j in range(i, n):
            v[j] = b[j]
            if i == j:
                v[j] += np.sign(b[j]) * np.sqrt(np.sum(b * b))

        Hi = get_householder(v)

        Ai = Hi @ Ai
        Q @= Hi

    R = Ai.copy()
    return Q, R

Q, R = get_qr(A)
display(pd.DataFrame(Q), pd.DataFrame(R))
```

	0	1	2
0	-0.818182	-0.095042	0.567050
1	0.545455	-0.440184	0.713244
2	0.181818	0.892864	0.411991

	0	1	2
0	-1.100000e+01	0.909091	1.454545
1	6.379434e-16	-8.010780	2.513499
2	-9.654674e-16	-0.030962	6.047034

```
np.allclose(Q @ R, A, eps)
```

```
True
```

Поиск собственных значений

```
Ak = A.copy()
Qk, Rk = get_qr(Ak)
iterations = 0

found = np.zeros(n, dtype=bool)
block_found = np.zeros(n, dtype=bool)
lambdas = np.zeros(n, dtype=complex)

def check_real_eigen(Ak, j):
    return eps >= np.sqrt(np.sum(Ak[j + 1:, j] ** 2))

while not all(found):
    found = np.zeros(n, dtype=bool)
    j = 0
    iterations += 1

    while j < n:
        if j == n - 1:
            lambdas[j] = Ak[j, j]
            found[j] = True
            block_found[j] = False
            j += 1
            continue

        if check_real_eigen(Ak, j):
            lambdas[j] = Ak[j, j]
            found[j] = True
            block_found[j] = False
            j += 1
            continue

        block = Ak[j:j+2, j:j+2]
```

```
        b = block[0, 0] + block[1, 1]
        c = block[0, 0] * block[1, 1] - block[0, 1] * block[1, 0]

        d = b * b - 4 * c
        if d > -eps:
            block_found[j] = False
            j += 1
            continue

        l1 = (b + np.emath.sqrt(d)) / 2
        l2 = (b - np.emath.sqrt(d)) / 2

        if (
            block_found[j]
            and np.abs(l2 - lambdas[j]) <= eps
            and np.abs(l1 - lambdas[j + 1]) <= eps
        ):
            found[j] = found[j + 1] = True
        block_found[j] = True

        lambdas[j] = l2
        lambdas[j + 1] = l1

        j += 2

    Ak = Rk @ Qk
    Qk, Rk = get_qr(Ak)

display(pd.DataFrame(lambdas))
print(f"Iterations: {iterations}")
```

	0
0	10.027222+ 0.000000j
1	3.986389- 6.096106j
2	3.986389+ 6.096106j

```
Iterations: 29
```

```
def complex_to_latex(z, tol=1e-10):
    a = z.real
    b = z.imag

    if abs(a) < tol:
        a = 0.0
    if abs(b) < tol:
        b = 0.0

    if b == 0:
        return f"{a:.3f}"

    if a == 0:
        if b == 1:
            return "i"
```

```
        if b == -1:
            return "-i"
        return f"{b:.3f}i"

    sign = "+" if b > 0 else "-"
    b_abs = abs(b)

    if abs(b_abs - 1) < tol:
        return f"{a:.3f} {sign} i"
    return f"{a:.3f} {sign} {b_abs:.3f}i"

lambdas = np.sort_complex(lambdas)
for i, lam in enumerate(lambdas, start=1):
    display(Math(rf"\lambda_{{{i}}} = {complex_to_latex(lam)}"))
```

$\lambda_1 = 3.986 - 6.096i$

$\lambda_2 = 3.986 + 6.096i$

$\lambda_3 = 10.027$

Валидация

```
eigenvalues, eigenvectors = np.linalg.eig(A)

eigenvalues = np.sort_complex(eigenvalues)
print(eigenvalues)
```

```
[ 3.98636265-6.09624525j  3.98636265+6.09624525j 10.0272747 +0.j        ]
```

```
np.allclose(lambdas, eigenvalues, eps)
```

```
True
```